

System Calls

System Call Explained: What, Why, How

- A program operates in one of the two modes: user mode or kernel mode.
- With kernel mode, a program has direct access to the underlying hardware and resources; whereas with user mode, it does not.
- The problem with kernel mode is that, when a program crashes, it can cause the entire system to crash or comes to halt. This is not the case with user mode.
- Thus, **System Calls** are software interfaces (mainly C programs) that a user program calls to the kernel (sometimes also called kernel calls), so that the program can use low level resources while avoiding the problem mentioned above with kernel mode.

System Call Details

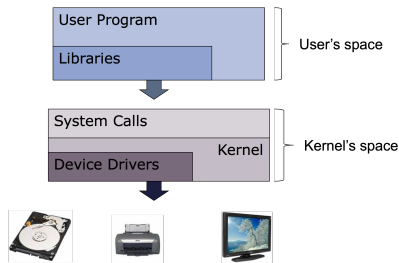
A system call is an expensive routine. Its key activities include:

- change user mode to kernel mode, known as **context switch**
- save all parameters of a user program to CPU
- load all the arguments data to CPU
- check file permission in case of read/write request
- execute the system call procedure
- after the system call, load all the saved parameters to CPU
- change from kernel mode back to user mode
- continue execution of user's program

Additionally, system call uses unbuffered I/O, which means the data gets written out (and the I/O gets done) to the output device as soon as your program issues the call/request to do so.

System Calls vs. Library Calls

- To reduce the overhead of system calls, systems such as Unix/Linux provide library functions, which are implemented on top of system calls.
- Unlike system calls, library functions use buffers (temporary memory storage), and run in user space.
- With these library functions, system call is only needed when the buffer size becomes full.



Types of System Calls

- File management system call:
create/delete file, open/close file, read/write file, get/set file attributes.
- Process control system call:
create/terminate/abort process, get/set process attributes, wait for time, wait for event, signal event, allocate/free memory.
- Device management system call:
request/release device, read/write console device, get/set device attributes, logically attach or detach devices
- Information maintenance system call: for information being shared between processes and operating systems.
get/set time or date, get/set system data, get/set process, file, or device attributes.
- Inter-process communications system call:
create/delete communication connection, send/receive messages, transfer status information, attach or detach remote devices

File Management System Calls

function prototypes for read file, write file, respectively.

```
#include <unistd.h>
ssize_t read(int fd, const void *buf, size_t nbytes);
ssize_t write(int fd, const void *buf, size_t nbytes);
```

fd The *file descriptor* is a non-negative integer value, obtained from a call to `open()`. The values 0, 1, 2 can also be given, representing standard input, standard output & standard error, respectively.

buf Points to a character array, with content to be written to (or read from) the file pointed to by `filedes`.

nbyte Specifies the number of bytes to be written to (or read from) the character array, `buf`, into the file pointed to by `filedes`.

The read/write function returns the number of bytes successfully read from or written into the file, respectively, which may at times be less than the specified `nbytes`. It returns -1 if an exceptional condition is encountered

System Call - Write File via Standard Devices

```
//write.c
#include <unistd.h>
#include <stdlib.h>

int main() {
    if ((write (1, "hope you're having fun\n", 22)) != 22)
        write (2, "error on file descriptor 1\n", 26);
    exit (0);
}
```

- A file descriptor is an index into the file description table, which hold pointers to "resources" such as files, terminal i/o, pipes, socket, devices. etc.
- It uses an unused lowest number between 0 and 1023 for a new resource.
- To the kernel, all open files are referred to by file descriptors.
- When we open an existing file or create a new file, the kernel returns a file descriptor to the process (i.e. the running program).
- We can read or write a file with the file descriptor obtained by system calls `open()` or `create()`, or the values can be given as mentioned before.

An In-Depth Example for File Descriptors

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(){
    int ifd, ofd;
    printf("process file descriptor table \n");
    printf("descriptor\t pointer\t descriptpion\n");
    printf("%d\t %p\t standard input\n", STDIN_FILENO, stdin);
    printf("%d\t %p\t standard output\n", STDOUT_FILENO, stdout);
    printf("%d\t %p\t standard error device\n", STDERR_FILENO, stderr);
    //create two files, one input and one output
    ifd=open("in.txt", O_RDONLY);
    ofd=open("out.txt", O_WRONLY);
    printf("%d\t %p\t in.txt\n", ifd, fdopen(ifd, "r")); //fdopen returns a pointer to file stream
    printf("%d\t %p\t out.txt\n", ofd, fdopen(ofd, "w"));

    return 0;
}
```

Output:

descriptor	pointer	descriptpion
0	0x1d79c43d8	standard input
1	0x1d79c4470	standard output
2	0x1d79c4508	standard error device
3	0x1d79c58a0	in.txt
4	0x1d79c5938	out.txt

System Call - Read/Write File via Standard Devices

```
//read.c
#include <unistd.h>
#include <stdlib.h>

int main() {
    char buffer[10];
    int nread = read(0, buffer, 10);

    if (nread == -1){
        write (2, "A read error\n", 10);
    }
    if ((write (1, buffer, nread))!= nread){
        write (2, "A write Error!\n", 14);
    }

    exit (0);
}
```

More Read/Write System Call Example

```
//readWrite.c
/* read and write byte by byte until empty (CTR-D)*/

#include <unistd.h>
#include <stdlib.h>

int main() {
    char b[1];
    int nread;
    while ((nread = read(0, b, 1)) > 0)
        write (1, b, nread);

    exit (0);
}
```

Using read/write System Calls to Copy Files

Copy standard input to standard output using read and write system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> /* STDIN_FILENO, STDOUT_FILENO */

/* buffer size effect the efficiency of the program */
#define BUFFER_SIZE 2
void err_sys(char *str) {
    printf ("%s",str);
    exit (1);
}
int main() {
    int nbyte;
    char buffer[BUFFER_SIZE];

    while ((nbyte = read(STDIN_FILENO, buffer, BUFFER_SIZE)) >0)
        if (write (STDOUT_FILENO, buffer, nbyte) != nbyte)
            err_sys ("Write Error");
    if (nbyte <0)
        err_sys("read Error");
    exit (0);
}
```

System Call - open() - Flag Definitions

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

// it returns a file descriptor, or -1 if error in opening the file.
// the mode defines file permissions;
// it is used only when a new file is created.
```

```
int open(const char *fname, int flags )
int open (const char *fname, int flags, mode_t mode);
```

The available flags as defined in `"/usr/include/fcntl.h"` are:

```
#define O_RDONLY      0      /* Open the file for reading only */
#define O_WRONLY      1      /* Open the file for writing only */
#define O_RDWR        2      /* Open the file for reading and writing*/
#define O_NDELAY       04     /* Non-blocking I/O */
#define O_APPEND       010    /* append (writes to the end) */
#define O_CREAT        00400  /*open with file creation (use third param) */
#define O_TRUNC        01000  /* open with truncation */
#define O_EXCL         02000  /* exclusive open */
```

System Call - open() - Mode Definition

```
#include <fcntl.h>

// it returns a file descriptor, or -1 if error in opening the file.
// the mode defines file permissions;
// it is used only when a new file is created.

int open(const char *fname, int flags )
int open (const char *fname, int flags, mode_t mode);
```

The available `mode_type` as defined in the header "`sys/stat.h`" are:

<code>S_IRUSR</code>	Read permission, owner; same as <code>S_IREAD</code> .
<code>S_IWUSR</code>	Write permission, owner; same as <code>S_IWRITE</code> .
<code>S_IXUSR</code>	Execute/search permission, owner.
<code>S_IRGRP</code>	Read permission, group.
<code>S_IWGRP</code>	Write permission, group.
<code>S_IXGRP</code>	Execute/search permission, group.
<code>S_IROTH</code>	Read permission, others.
<code>S_IWOTH</code>	Write permission, others.
<code>S_IXOTH</code>	Execute/search permission, others.

Or we can use octal form 0777, 0755, 0555, ...

open() - Code Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

static char message[] = "Hello, world";

int main() {
    int fd;
    char buffer[80];
    /* open for rw and create exclusive open, ceate as rw----- */
    fd = open("datafile.dat", O_RDWR | O_CREAT | O_EXCL, S_IREAD | S_IWRITE);
    printf ("fd = %d\n", fd);
    if (fd != -1) {
        printf("datafile.dat opened for read/write access\n");
        write(fd, message, sizeof(message));
        close (fd);
    }
    else
        printf("*** datafile.dat already exists ***\n");
    exit (0);
}
```

open() - Code Example with Verification

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
static char message[] = "Hello, world";

int main() {
    int fd;
    char buffer[80];
    /* open for rw and create exclusive open, ceate as rw----- */
    fd = open("datafile.dat", O_RDWR | O_CREAT | O_EXCL, S_IRREAD | S_IWRITE);
    printf ("fd = %d\n", fd);
    if (fd != -1) {
        printf("datafile.dat opened for read/write access\n");
        write(fd, message, sizeof(message));
        lseek(fd, 0, SEEK_SET); /* go back to the beginning of the file */
        if (read(fd, buffer, sizeof(message)) == sizeof(message))
            printf("\n%s" was written to datafile.dat\n", buffer);
        else
            printf("*** error reading datafile.dat ***\n");
        close (fd);
    }
    else
        printf("*** datafile.dat already exists ***\n");
    exit (0);
}
```

System Calls - creat() and close() Prototype

A new file can also be created by using the `creat()` system call.

```
#include <fcntl.h>

// it returns a file descriptor. On error, -1 is returned.
// the mode defines file permissions;

int creat(const char *fname, mode_t mode);
```

Any opened file is closed by a system call `close()`, which closes a file descriptor, so that it no longer refers to any file and may be reused.

```
#include <fcntl.h>

// returns zero on success. On error, -1 is returned.

int close (int fd);
```


Copy File Using System Calls

- Read contents of a file one byte at a time and write to a file, or
- Read contents of a file and save into a buffer, then write to a file.

Code Example - Copy File One Byte A Time

```
/*copyfile.c: read a character by character from a copyfile.c
and write to out.txt file */
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/stat.h>

int main() {
    int InFileDes, OutFileDes;
    char Ach;
    /* open file for read only */
    InFileDes = open("copyfile.c", O_RDONLY);
    /* open file for write only or create only,
    created file permission mode with rw----- */
    OutFileDes = open ("out.txt", O_WRONLY|O_CREAT, S_IRUSR|S_IWUSR);
    while (read(InFileDes, &Ach, 1) == 1)
        write(OutFileDes, &Ach, 1);
    close (InFileDes);
    close (OutFileDes);
    exit (0);
}
```

Code Example - Copy Two Files One Byte A Time

```
/* copyfile2.c: read a character by character from two
files and write to updown.txt */
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

int main() {
    int InFileDes, InFileDes1, OutFileDes;
    char Ach, Ach1;
    /* open file for read only */
    InFileDes = open("upper.txt", O_RDONLY);
    InFileDes1 = open("lower.txt", O_RDONLY);
    /* open file for write only or create only, created file mode will rw----- */
    OutFileDes = open ("updown.txt", O_WRONLY|O_CREAT, S_IRUSR|S_IWUSR);
    while ( (read(InFileDes, &Ach, 1) == 1)&& (read(InFileDes1, &Ach1, 1))) {
        write(OutFileDes, &Ach, 1);
        write(OutFileDes, &Ach1, 1);
    }
    close (InFileDes);
    close (InFileDes1);
    close (OutFileDes);
    exit (0);
}
```

Code Example - Copy File via Buffer

```
/*copyBfile.c: read data and save into a buffer
and write to updown.txt */
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>
#define BUFFER_SIZE 4096
void err_sys(char *str) {
    printf ("%s",str);
    exit (1);
}
int main() {
    int InFileDes,OutFileDes;
    int nbyte;
    char buffer[BUFFER_SIZE];
    /* open file for read only */
    InFileDes = open("copyfile.c", O_RDONLY);
    /* open file for write only or create only, created file mode will rw-----*/
    OutFileDes = open ("copyfile.txt", O_WRONLY|O_CREAT, S_IRUSR|S_IWUSR);
    while ((nbyte = read(InFileDes, buffer, BUFFER_SIZE)) >0)
        if (write (OutFileDes, buffer, nbyte) !=nbyte)
            err_sys("Write Error");
        if (nbyte <0)
            err_sys("Read Error");

    close (InFileDes);
    close (OutFileDes);
    exit (0);
}
```